Chapter 8. Scripting, Compiling, and Annotation Processing

In this chapter

- 8.1 Scripting for the Java Platform
- 8.2 The Compiler API
- 8.3 Using Annotations
- 8.4 Annotation Syntax
- 8.5 Standard Annotations
- 8.6 Source-Level Annotation Processing
- 8.7 Bytecode Engineering

This chapter introduces three techniques for processing code. The scripting API lets you invoke code in a scripting language such as JavaScript or Groovy. You can use the compiler API when you want to compile Java code inside your application. Annotation processors operate on Java source or class files that contain annotations. As you will see, there are many applications for annotation processing, ranging from simple diagnostics to "bytecode engineering"—the insertion of bytecodes into class files or even running programs.

8.1 Scripting for the Java Platform

A scripting language is a language that avoids the usual edit/compile/link/run cycle by interpreting the program text at runtime. Scripting languages have a number of advantages:

- Rapid turnaround, encouraging experimentation

- Changing the behavior of a running program
- Enabling customization by program users

On the other hand, most scripting languages lack features that are beneficial for programming complex applications, such as strong typing, encapsulation, and modularity.

It is therefore tempting to combine the advantages of scripting and traditional languages. The scripting API lets you do just that for the Java platform. It enables you to invoke scripts written in JavaScript, Groovy, Ruby, and even exotic languages such as Scheme and Haskell, from a Java program. For example, the Renjin project (

`www.renjin.org`

) provides a Java implementation of the R programming language, which is commonly used for statistical programming, together with an "engine" of the scripting API.

In the following sections, I'll show you how to select an engine for a particular language, how to execute scripts, and how to make use of advanced features that some scripting engines offer.

8.1.1 Getting a Scripting Engine

A scripting engine is a library that can execute scripts in a particular language. When the virtual machine starts, it discovers the available scripting engines. To enumerate them, construct a

`ScriptEngineManager`

and invoke the

`getEngineFactories`

method. You can ask each engine factory for the supported engine names, MIME types, and file extensions. **Table 8.1** shows typical values.

Table 8.1 Properties of Scripting Engine Factories

Engine	Names	MIME types	Extensions
Rhino (JavaScript)	rhino, Rhino, JavaScript, javascript	`application/javascript`, `application/ecmascript`, `text/javascript`, `text/ecmascript`	`js`
Groovy	groovy	None	`groovy`
Renjin	Renjin	`text/x-R`	`R`, `r`, `S`, `s`

Usually, you know which engine you need, and you can simply request it by name, MIME type, or extension. For example:

Click here to view code image

```
var manager = new ScriptEngineManager();
ScriptEngine engine = manager.getEngineByName("javascript");
```

You need to provide the JAR files that implement the script engine on the classpath. (The Oracle JDK used to contain a JavaScript engine, but it has been removed in Java 15.)

`javax.script.ScriptEngineManager`

6

- `List<ScriptEngineFactory> getEngineFactories()`
  gets a list of all discovered engine factories.
- `ScriptEngine getEngineByName(String name)`
- `ScriptEngine getEngineByExtension(String extension)`
- `ScriptEngine getEngineByMimeType(String mimeType)`
  get the script engine with the given name, script file extension, or MIME type.

`javax.script.ScriptEngineFactory`

6

- `List<String> getNames()`
- `List<String> getExtensions()`
- `List<String> getMimeTypes()`

get the names, script file extensions, and MIME types under which this factory is known.

8.1.2 Script Evaluation and Bindings

Once you have an engine, you can call a script simply by invoking

Click here to view code image

```
Object result = engine.eval(scriptString);
```

If the script is stored in a file, open a

```
Reader
```

and call

Click here to view code image

```
Object result = engine.eval(reader);
```

You can invoke multiple scripts on the same engine. If one script defines variables, functions, or classes, most scripting engines retain the definitions for later use. For example,

Click here to view code image

```
engine.eval("n = 1728");
Object result = engine.eval("n + 1");
```

will return

```
1729
```

.

 **Note**

To find out whether it is safe to concurrently execute scripts in multiple threads, call

Click here to view code image

```
Object param = factory.getParameter("THREADING");
```

The returned value is one of the following:

- `null`
  : Concurrent execution is not safe.
- `"MULTITHREADED"`
  : Concurrent execution is safe. Effects from one thread might be visible from another thread.

- `"THREAD-ISOLATED"`
  : In addition to `"MULTITHREADED"`, different variable bindings are maintained for each thread.

- `"STATELESS"`
  : In addition to `"THREAD-ISOLATED"`, scripts do not alter variable bindings.

You will often want to add variable bindings to the engine. A binding consists of a name and an associated Java object. For example, consider these statements:

Click here to view code image

```
engine.put("k", 1728);
Object result = engine.eval("k + 1");
```

The script code reads the definition of `k` from the bindings in the "engine scope." This is particularly important because most scripting languages can access Java objects, often with a syntax that is simpler than the Java syntax. For example,

Click here to view code image

```
engine.put("b", new JButton());
engine.eval("b.text = 'Ok'");
```

Conversely, you can retrieve variables that were bound by scripting statements:

Click here to view code image

```
engine.eval("n = 1728");
Object result = engine.get("n");
```

In addition to the engine scope, there is also a global scope. Any bindings that you add to the `ScriptEngineManager` are visible to all engines.

Instead of adding bindings to the engine or global scope, you can collect them in an object of type `Bindings` and pass it to the `eval` method:

Click here to view code image

```
Bindings scope = engine.createBindings();
scope.put("b", new JButton());
engine.eval(scriptString, scope);
```

This is useful if a set of bindings should not persist for future calls to the `eval` method.



Note

You might want to have scopes other than the engine and global scopes. For example, a web container might need request and session scopes. However, then you are on your own. You will need to write a class that implements the `ScriptContext` interface, managing a collection of scopes. Each scope is identified by an integer number, and scopes with lower numbers should be searched first. (The standard library provides a `SimpleScriptContext` class, but it only holds global and engine scopes.)

- `Object eval(String script)`

- `Object eval(Reader reader)`
- `Object eval(String script, Bindings bindings)`
- `Object eval(Reader reader, Bindings bindings)`

evaluate the script given by the string or reader, subject to the given bindings.

- `Object get(String key)`
- `void put(String key, Object value)`

get or put a binding in the engine scope.

- `Bindings createBindings()`

creates an empty

`Bindings`

object suitable for this engine.

`javax.script.ScriptEngineManager`

6

- `Object get(String key)`
- `void put(String key, Object value)`

get or put a binding in the global scope.

`javax.script.Bindings`

6

- `Object get(String key)`

- `void put(String key, Object value)`

  get or put a binding into the scope represented by this `Bindings` object.

8.1.3 Redirecting Input and Output

You can redirect the standard input and output of a script by calling the `setReader` and `setWriter` methods of the script context. For example,

Click here to view code image

```
var writer = new StringWriter();
engine.getContext().setWriter(new PrintWriter(writer, true));
```

Any output written with the JavaScript `print` or `println` functions is sent to

`writer`

.

The

`setReader`

and

`setWriter`

methods only affect the scripting engine's standard input and output sources. For example, if you execute the JavaScript code

Click here to view code image

```
println("Hello");
java.lang.System.out.println("World");
```

only the first output is redirected.

The Rhino engine does not have the notion of a standard input source. Calling

`setReader`

has no effect.

---

javax.script.ScriptEngine

- `ScriptContext getContext()`

gets the default script context for this engine.

---

javax.script.ScriptContext

6

- Reader getReader()
- void setReader(Reader reader)
- Writer getWriter()
- void setWriter(Writer writer)
- Writer getErrorWriter()
- void setErrorWriter(Writer writer)

get or set the reader for input or writer for normal or error output.

8.1.4 Calling Scripting Functions and Methods

With many script engines, you can invoke a function in the scripting language without having to evaluate the actual script code. This is useful if you allow users to implement a service in a scripting language of their choice.

The script engines that offer this functionality implement the

Invocable

interface. In particular, the Rhino engine implements

Invocable

.

To call a function, call the

`invokeFunction`

method with the function name, followed by the function parameters:

Click here to view code image

```
// Define greet function in JavaScript
engine.eval("function greet(how, whom) { return how + ', ' + whom + '!' }");
// Call the function with arguments "Hello", "World"
result = ((Invocable) engine).invokeFunction("greet", "Hello", "World");
```

If the scripting language is object-oriented, call

`invokeMethod`

:

Click here to view code image

```
// Define Greeter class in JavaScript
engine.eval("function Greeter(how) { this.how = how }");
engine.eval("Greeter.prototype.welcome = "
   + " function(whom) { return this.how + ', ' + whom + '!' }");

// Construct an instance
Object yo = engine.eval("new Greeter('Yo')");
```

```
// Call the welcome method on the instance
result = ((Invocable) engine).invokeMethod(yo, "welcome", "World");
```

 **Note**

Rhino does not support the modern JavaScript class syntax. For more information on how to define classes in JavaScript the old-fashioned way, see *JavaScript—The Good Parts* by Douglas Crockford (O'Reilly, 2008).

 **Note**

If the script engine does not implement the `Invocable` interface, you might still be able to call a method in a language-independent way. The `getMethodCallSyntax` method of the `ScriptEngineFactory` interface produces a string that you can pass to the `eval` method. For this, however, all method parameters must be bound to names, whereas `invokeMethod` can be called with arbitrary values.

You can go a step further and ask the scripting engine to implement a Java interface. Then you can call scripting functions and methods with the Java method call syntax.

The details depend on the scripting engine, but typically you need to supply a function for each method of the interface. For example, consider a Java interface

```
public interface Greeter
{
  String welcome(String whom);
}
```

If you define a global function with the same name in Rhino, you can call it through this interface:

Click here to view code image

```
// Define welcome function in JavaScript
engine.eval("function welcome(whom) { return 'Hello, ' + whom + '!' }");
// Get a Java object and call a Java method
Greeter g = ((Invocable) engine).getInterface(Greeter.class);
result = g.welcome("World");
```

In an object-oriented scripting language, you can access a script class through a matching Java interface. For example, here is how to call an object of the JavaScript

```
Greeter
```

class with Java syntax:

Click here to view code image

```
Greeter g = ((Invocable) engine).getInterface(yo, Greeter.class);
result = g.welcome("World");
```

In summary, the

`Invocable`

interface is useful if you want to call scripting code from Java without worrying about the scripting language syntax.

---

javax.script.Invocable

6

- `Object invokeFunction(String name, Object... parameters)`
- `Object invokeMethod(Object implicitParameter, String name, Object... explicitParameters)`

  invoke the function or method with the given name, passing the given parameters.
- `<T> T getInterface(Class<T> iface)`

  returns an implementation of the given interface, implementing the methods with functions in the scripting engine.
- `<T> T getInterface(Object implicitParameter, Class<T> iface)`

  returns an implementation of the given interface, implementing the methods with the methods of the given object.

8.1.5 Compiling a Script

Some scripting engines can compile scripting code into an intermediate form for efficient execution. Those engines implement the

`Compilable`

interface. The following example shows how to compile and evaluate code contained in a script file:

Click here to view code image

```
var reader = new FileReader("myscript.js");
CompiledScript script = null;
if (engine implements Compilable)
   script = ((Compilable) engine).compile(reader);
```

Once the script is compiled, you can execute it. The following code executes the compiled script if compilation was successful, or the original script if the engine didn't support compilation:

```
if (script != null)
   script.eval();
else
   engine.eval(reader);
```

Of course, it only makes sense to compile a script if you need to execute it repeatedly.

---

javax.script.Compilable

6

- `CompiledScript compile(String script)`
- `CompiledScript compile(Reader reader)`

  compile the script given by a string or reader.

`javax.script.CompiledScript`

6

- `Object eval()`
- `Object eval(Bindings bindings)`

  evaluate this script.

8.1.6 An Example: Scripting GUI Events

To illustrate the scripting API, we will write a sample program that allows users to specify event handlers in a scripting language of their choice.

Have a look at the program in **Listing 8.1** that adds scripting to an arbitrary frame class. By default it reads the `ButtonFrame` class in **Listing 8.2**, which is similar to the event handling demo in Volume I, with two differences:

- Each component has its `name`

property set.
- There are no event handlers.

The event handlers are defined in a property file. Each property definition has the form

Click here to view code image

```
componentName.eventName = scriptCode
```

For example, if you choose to use JavaScript, supply the event handlers in a file

```
js.properties
```

, like this:

Click here to view code image

```
yellowButton.action=panel.background = java.awt.Color.YELLOW
blueButton.action=panel.background = java.awt.Color.BLUE
redButton.action=panel.background = java.awt.Color.RED
```

The companion code also has files for Groovy and R.

The program starts by loading an engine for the language specified on the command line. If no language is specified, we use JavaScript.

We then process a script

`init.language`

if it is present. This is useful for languages such as R which need some initializations that we do not want to include in every event handler script.

Next, we recursively traverse all child components and add the bindings (

`name`

,

`object`

) into a map of components. Then we add the bindings to the engine.

Next, we read the file

`language.properties`

. For each property, we synthesize an event handler proxy that causes the script code to be executed. The details are a bit technical. You might want to read the section on proxies in Volume I, **Chapter 6**, if you want to follow the implementation in detail. The essential part, however, is that each event handler calls

```
engine.eval(scriptCode);
```

Let us look at the

`yellowButton`

in more detail. When the line

Click here to view code image

```
yellowButton.action=panel.background = java.awt.Color.YELLOW
```

is processed, we find the

`JButton`

component with the name

`"yellowButton"`

. We then attach an

`ActionListener`

with an

`actionPerformed`

method that executes the script

Click here to view code image

```
panel.background = java.awt.Color.YELLOW
```

if the scripting is done with Rhino.

The engine contains a binding that binds the name

`"panel"`

to the

`JPanel`

object. When the event occurs, the

`setBackground`

method of the panel is executed, and the color changes.

If you use Java 15 or beyond, the classpath must contain a JavaScript engine such as Rhino (

`https://github.com/mozilla/rhino`

). Start the program like this:

Click here to view code image

```
java -classpath .:rhino-version.jar:rhino-engine-version.jar ScriptTest
```

For the Groovy handlers, use

Click here to view code image

```
java -classpath .:groovy/lib/\* ScriptTest groovy
```

Here,

groovy

is the directory into which you installed Groovy.

For the Renjin implementation of R, include the JAR files for Renjin Studio and the Renjin script engine on the classpath. Both are available at

www.renjin.org/downloads.html

.

This application demonstrates how to use scripting for Java GUI programming. One could go a step further and describe the GUI with an XML file, as you have seen in **Chapter 3**. Then our program would become an interpreter for GUIs that have their visual presentation defined in XML and behavior defined in a scripting language. Note the similarity to a dynamic HTML page or a dynamic server-side scripting environment.

Listing 8.1

script/ScriptTest.java

Click here to view code image

```
 1 package script;
 2
 3 import java.awt.*;
 4 import java.beans.*;
 5 import java.io.*;
 6 import java.lang.reflect.*;
 7 import java.nio.charset.*;
 8 import java.util.*;
 9 import javax.script.*;
10 import javax.swing.*;
11
12 /**
13  * @version 1.04 2021-06-17
14  * @author Cay Horstmann
```

```
15 */
16 public class ScriptTest
17 {
18    public static void main(String[] args)
19    {
20       EventQueue.invokeLater(() ->
21          {
22             try
23             {
24                var manager = new ScriptEngineManager();
25                String language;
26                if (args.length == 0)
27                {
28                   System.out.println("Available factories: ");
29                   for (ScriptEngineFactory factory : manager.getEngineFactories())
30                      System.out.println(factory.getEngineName());
31
32                   language = "nashorn";
33                }
34                else language = args[0];
35
36                final ScriptEngine engine = manager.getEngineByName(language);
37                if (engine == null)
38                {
39                   System.err.println("No engine for " + language);
40                   System.exit(1);
41                }
42
43                final String frameClassName
44                   = args.length < 2 ? "buttons1.ButtonFrame" : args[1];
45
46                var frame
47                   = (JFrame) Class.forName(frameClassName).getConstructor().newInstance();
```

```
48              InputStream in = frame.getClass().getResourceAsStream("init." + language);
49              if (in != null) engine.eval(
50                 new InputStreamReader(in, StandardCharsets.UTF_8));
51              var components = new HashMap<String, Component>();
52              getComponentBindings(frame, components);
53              components.forEach((name, c) -> engine.put(name, c));
54
55              var events = new Properties();
56              in = frame.getClass().getResourceAsStream(language + ".properties");
57              events.load(
58                 new InputStreamReader(in, StandardCharsets.UTF_8));
59
60              for (Object e : events.keySet())
61              {
62                 String[] s = ((String) e).split("\\.");
63                 addListener(s[0], s[1], (String) events.get(e), engine, components);
64              }
65              frame.setTitle("ScriptTest");
66              frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
67              frame.setVisible(true);
68           }
69           catch (ReflectiveOperationException | IOException
70                 | ScriptException | IntrospectionException e)
71           {
72             e.printStackTrace();
73           }
74        });
75    }
76
77    /**
78     * Gathers all named components in a container.
79     * @param c the component
80     * @param namedComponents a map into which to enter the component names and components
```

```
81      */
82     private static void getComponentBindings(Component c,
83           Map<String, Component> namedComponents)
84     {
85        String name = c.getName();
86        if (name != null) { namedComponents.put(name, c); }
87        if (c instanceof Container container)
88        {
89           for (Component child : container.getComponents())
90              getComponentBindings(child, namedComponents);
91        }
92     }
93
94    /**
95     * Adds a listener to an object whose listener method executes a script.
96     * @param beanName the name of the bean to which the listener should be added
97     * @param eventName the name of the listener type, such as "action" or "change"
98     * @param scriptCode the script code to be executed
99     * @param engine the engine that executes the code
100    * @param bindings the bindings for the execution
101    * @throws IntrospectionException
102    */
103   private static void addListener(String beanName, String eventName, final String scriptCode,
104         ScriptEngine engine, Map<String, Component> components)
105         throws ReflectiveOperationException, IntrospectionException
106   {
107     Object bean = components.get(beanName);
108     EventSetDescriptor descriptor = getEventSetDescriptor(bean, eventName);
109     if (descriptor == null) return;
110     descriptor.getAddListenerMethod().invoke(bean,
111        Proxy.newProxyInstance(null, new Class[] { descriptor.getListenerType() },
112           (proxy, method, args) ->
113              {
```

```
               engine.eval(scriptCode);
               return null;
            }));
  }

  private static EventSetDescriptor getEventSetDescriptor(Object bean, String eventName)
        throws IntrospectionException
  {
     for (EventSetDescriptor descriptor : Introspector.getBeanInfo(bean.getClass())
           .getEventSetDescriptors())
        if (descriptor.getName().equals(eventName)) return descriptor;
     return null;
   }
 }
```

Listing 8.2

buttons1/ButtonFrame.java

Click here to view code image

```
 1 package buttons1;
 2
 3 import javax.swing.*;
 4
 5 /**
 6  * A frame with a button panel.
 7  * @version 1.00 2007-11-02
 8  * @author Cay Horstmann
 9  */
```

```
10 public class ButtonFrame extends JFrame
11 {
12    private static final int DEFAULT_WIDTH = 300;
13    private static final int DEFAULT_HEIGHT = 200;
14
15    private JPanel panel;
16    private JButton yellowButton;
17    private JButton blueButton;
18    private JButton redButton;
19
20    public ButtonFrame()
21    {
22       setSize(DEFAULT_WIDTH, DEFAULT_HEIGHT);
23
24       panel = new JPanel();
25       panel.setName("panel");
26       add(panel);
27
28       yellowButton = new JButton("Yellow");
29       yellowButton.setName("yellowButton");
30       blueButton = new JButton("Blue");
31       blueButton.setName("blueButton");
32       redButton = new JButton("Red");
33       redButton.setName("redButton");
34
35       panel.add(yellowButton);
36       panel.add(blueButton);
37       panel.add(redButton);
38    }
39 }
```

8.2 The Compiler API

There are quite a few tools that need to compile Java code. Obviously, development environments and programs that teach Java programming are among them, as well as testing and build automation tools. Another example is the processing of JavaServer Pages—web pages with embedded Java statements.

8.2.1 Invoking the Compiler

It is very easy to invoke the compiler. Here is a sample call:

Click here to view code image

```
JavaCompiler compiler = ToolProvider.getSystemJavaCompiler();
OutputStream outStream = . . .;
OutputStream errStream = . . .;
int result = compiler.run(null, outStream, errStream,
   "-sourcepath", "src", "Test.java");
```

A `result` value of `0` indicates successful compilation.

The compiler sends its output and error messages to the provided streams. You can set these parameters to `null`

, in which case `System.out` and `System.err` are used. The first parameter of the `run` method is an input stream. As the compiler takes no console input, you can always leave it as `null`. (The `run` method is inherited from a generic `Tool` interface, which allows for tools that read input.)

The remaining parameters of the `run` method are the arguments that you would pass to `javac` if you invoked it on the command line. These can be options or file names.

8.2.2 Launching a Compilation Task

You can have more control over the compilation process with a `CompilationTask` object. This can be useful if you want to supply source from string, capture class files in memory, or process the error and warning messages.

To obtain a

`CompilationTask`

, start with a

`compiler`

object as in the preceding section. Then call

Click here to view code image

```
JavaCompiler.CompilationTask task = compiler.getTask(
  errorWriter, // Uses System.err if null
  fileManager, // Uses the standard file manager if null
  diagnostics, // Uses System.err if null
  options, // null if no options
  classes, // For annotation processing; null if none
  sources);
```

The last three arguments are

`Iterable`

instances. For example, a sequence of options might be specified as

Click here to view code image

```
Iterable<String> options = List.of("-d", "bin");
```

The `sources` parameter is an `Iterable` of `JavaFileObject` instances. If you want to compile disk files, get a `StandardJavaFileManager` and call its `getJavaFileObjects` method:

Click here to view code image

```
StandardJavaFileManager fileManager = compiler.getStandardFileManager(null, null, null);
Iterable<JavaFileObject> sources
   = fileManager.getJavaFileObjectsFromStrings(List.of("File1.java", "File2.java"));
JavaCompiler.CompilationTask task = compiler.getTask(
   null, null, null, options, null, sources);
```



Note

The `classes`

parameter is only used for annotation processing. In that case, you also need to call

```
task.processors(annotationProcessors)
```

with a list of

```
Processor
```

objects. See **Section 8.6**, "**Source-Level Annotation Processing**," on p. **488** for an example of annotation processing.

The

```
getTask
```

method returns the task object but does not yet start the compilation process. The

```
CompilationTask
```

class extends

```
Callable<Boolean>
```

. You can pass it to an

```
ExecutorService
```

for concurrent execution, or you can just make a synchronous call:

```
Boolean success = task.call();
```

8.2.3 Capturing Diagnostics

To listen to error messages, install a

```
DiagnosticListener
```

. The listener receives a

`Diagnostic`

object whenever the compiler reports a warning or error message. The

`DiagnosticCollector`

class implements this interface. It simply collects all diagnostics so that you can iterate through them after the compilation is complete.

Click here to view code image

```
DiagnosticCollector<JavaFileObject> collector = new DiagnosticCollector<>();
compiler.getTask(null, fileManager, collector, null, null, sources).call();
for (Diagnostic<? extends JavaFileObject> d : collector.getDiagnostics())
{
  System.out.println(d);
}
```

A

`Diagnostic`

object contains information about the problem location (including file name, line number, and column number) as well as a human-readable description.

You can also install a

`DiagnosticListener`

to the standard file manager, in case you want to trap messages about missing files:

Click here to view code image

```
StandardJavaFileManager fileManager
   = compiler.getStandardFileManager(diagnostics, null, null);
```

8.2.4 Reading Source Files from Memory

If you generate source code on the fly, you can have it compiled from memory, without having to save files to disk. Use this class to hold the code:

Click here to view code image

```
public class StringSource extends SimpleJavaFileObject
{
   private String code;
   StringSource(String name, String code)
   {
      super(URI.create("string:///" + name.replace('.','/') + ".java"), Kind.SOURCE);
      this.code = code;
    }

   public CharSequence getCharContent(boolean ignoreEncodingErrors)
   {
      return code;
   }
}
```

Then generate the code for your classes and give the compiler a list of

`StringSource`

objects:

Click here to view code image

```
List<StringSource> sources = List.of(
  new StringSource(className1, class1CodeString), . . .);
task = compiler.getTask(null, fileManager, diagnostics, null, null, sources);
```

8.2.5 Writing Byte Codes to Memory

If you compile classes on the fly, there is no need to save the class files to disk. You can save them to memory and load them right away.

First, here is a class for holding the bytes:

Click here to view code image

```
public class ByteArrayClass extends SimpleJavaFileObject
{
  private ByteArrayOutputStream out;

  ByteArrayClass(String name)
  {
    super(URI.create("bytes:///" + name.replace('.','/') + ".class"), Kind.CLASS);
  }
```

```
  public byte[] getCode()
  {
    return out.toByteArray();
  }

  public OutputStream openOutputStream() throws IOException
  {
    out = new ByteArrayOutputStream();
    return out;
  }
}
```

Next, you need to configure the file manager to use these classes for output:

Click here to view code image

```
List<ByteArrayClass> classes = new ArrayList<>();
StandardJavaFileManager stdFileManager

  = compiler.getStandardFileManager(null, null, null);
JavaFileManager fileManager
  = new ForwardingJavaFileManager<JavaFileManager>(stdFileManager)
    {
      public JavaFileObject getJavaFileForOutput(Location location,
          String className, Kind kind, FileObject sibling)
          throws IOException
      {
        if (kind == Kind.CLASS)
```

```
        {
          ByteArrayClass outfile = new ByteArrayClass(className);
          classes.add(outfile);
          return outfile;
        }
        else
          return super.getJavaFileForOutput(location, className, kind, sibling);
      }
    };
```

To load the classes, you need a class loader (see **Chapter 10**):

Click here to view code image

```
public class ByteArrayClassLoader extends ClassLoader
{
  private Iterable<ByteArrayClass> classes;

  public ByteArrayClassLoader(Iterable<ByteArrayClass> classes)
  {
    this.classes = classes;
  }

  public Class<?> findClass(String name) throws ClassNotFoundException
  {
    for (ByteArrayClass cl : classes)
    {
      if (cl.getName().equals("/" + name.replace('.','/') + ".class"))
      {
```

```
            byte[] bytes = cl.getCode();
            return defineClass(name, bytes, 0, bytes.length);
          }
        }
        throw new ClassNotFoundException(name);
      }
    }
```

After compilation has finished, call the

```
Class.forName
```

method with that class loader:

Click here to view code image

```
ByteArrayClassLoader loader = new ByteArrayClassLoader(classes);
Class<?> cl = Class.forName(className, true, loader);
```

8.2.6 An Example: Dynamic Java Code Generation

In the JSP technology for dynamic web pages, you can mix HTML with snippets of Java code, for example:

Click here to view code image

```
<p>The current date and time is <b><%= new java.util.Date() %></b>.</p>
```

The JSP engine dynamically compiles the Java code into a servlet. In our sample application, we use a simpler example and generate dynamic Swing code instead. The idea is that you use a GUI builder to lay out the components in a frame and specify the behavior of the components in an external file. **Listing 8.4** shows a very simple example of a frame class, and **Listing 8.5** shows the code for the button actions. Note that the constructor of the frame class calls an abstract method

`addEventHandlers`

. Our code generator will produce a subclass that implements the

`addEventHandlers`

method, adding an action listener for each line in the

`action.properties`

file. (I leave it as the proverbial exercise to the reader to extend the code generation to other event types.)

We place the subclass into a package with the name

`x`

, which we hope is not used anywhere else in the program. The generated code has the form

Click here to view code image

```
package x;
public class Frame extends SuperclassName
{
  protected void addEventHandlers()
  {
    componentName1.addActionListener(event ->
      {
        code for event handler1
      });
    // repeat for the other event handlers . . .
```

```
  }
}
```

The `buildSource` method in the program of **Listing 8.3** builds up this code and places it into a `StringSource` object. That object is passed to the Java compiler.

As described in the preceding section, we use a `ForwardingJavaFileManager` that constructs a `ByteArrayClass` object for every compiled class. These objects capture the class files generated when the `x.Frame` class is compiled. The method adds each file object to a list before returning it so that we can locate the bytecodes later.

After compilation, we use the class loader from the preceding section to load the classes stored in this list. Then, we construct and display the application's frame class.

Click here to view code image

```
var loader = new ByteArrayClassLoader(classFileObjects);
var frame = (JFrame) loader.loadClass("x.Frame").getConstructor().newInstance();
frame.setVisible(true);
```

When you click the buttons, the background color changes in the usual way. To see that the actions are dynamically compiled, change one of the lines in

`action.properties`

, for example, like this:

Click here to view code image

```
yellowButton=panel.setBackground(java.awt.Color.YELLOW); yellowButton.setEnabled(false);
```

Run the program again. Now the Yellow button is disabled after you click it. Also, have a look at the code directories. You will not find any source or class files for the classes in the

`x`

package. This example demonstrates how you can use dynamic compilation with in-memory source and class files.

Listing 8.3

compiler/CompilerTest.java

Click here to view code image

```
1   package compiler;
2
3   import java.awt.*;
4   import java.io.*;
5   import java.nio.charset.*;
6   import java.nio.file.*;
```

```
 7  import java.util.*;
 8  import java.util.List;
 9
10  import javax.swing.*;
11  import javax.tools.*;
12  import javax.tools.JavaFileObject.*;
13
14  /**
15   * @version 1.10 2018-05-01
16   * @author Cay Horstmann
17   */
18  public class CompilerTest
19  {
20     public static void main(final String[] args)
21           throws IOException, ReflectiveOperationException
22     {
23        JavaCompiler compiler = ToolProvider.getSystemJavaCompiler();
24
25        var classFileObjects = new ArrayList<ByteArrayClass>();
26
27        var diagnostics = new DiagnosticCollector<JavaFileObject>();
28
29        JavaFileManager fileManager = compiler.getStandardFileManager(diagnostics, null, null)
30        fileManager = new ForwardingJavaFileManager<JavaFileManager>(fileManager)
31           {
32           public JavaFileObject getJavaFileForOutput(Location location,
33                 String className, Kind kind, FileObject sibling) throws IOException
34           {
35              if (kind == Kind.CLASS)
36              {
37                 var fileObject = new ByteArrayClass(className);
38                 classFileObjects.add(fileObject);
39                 return fileObject;
```

```
            }
            else return super.getJavaFileForOutput(location, className, kind, sibling);
         }
      };


   String frameClassName = args.length == 0 ? "buttons2.ButtonFrame" : args[0];
   //compiler.run(null, null, null, frameClassName.replace(".", "/") + ".java");

   StandardJavaFileManager fileManager2 = compiler.getStandardFileManager(null, null, null);
   var sources = new ArrayList<JavaFileObject>();
   for (JavaFileObject o : fileManager2.getJavaFileObjectsFromStrings(
         List.of(frameClassName.replace(".", "/") + ".java")))
   sources.add(o);

   JavaFileObject source = buildSource(frameClassName);
   JavaCompiler.CompilationTask task = compiler.getTask(null, fileManager, diagnostics,
      null, null, List.of(source));
   Boolean result = task.call();

   for (Diagnostic<? extends JavaFileObject> d : diagnostics.getDiagnostics())
      System.out.println(d.getKind() + ": " + d.getMessage(null));
   fileManager.close();
   if (!result)
   {
      System.out.println("Compilation failed.");
      System.exit(1);
   }

   var loader = new ByteArrayClassLoader(classFileObjects);
   var frame = (JFrame) loader.loadClass("x.Frame").getConstructor().newInstance();

   EventQueue.invokeLater(() ->
```

```
73      {
74         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
75         frame.setTitle("CompilerTest");
76         frame.setVisible(true);
77       });
78   }
79
80   /*
81    * Builds the source for the subclass that implements the addEventHandlers method.
82    * @return a file object containing the source in a string builder
83    */
84   static JavaFileObject buildSource(String superclassName)
85         throws IOException, ClassNotFoundException
86   {
87      var builder = new StringBuilder();
88      builder.append("package x;\n\n");
89      builder.append("public class Frame extends " + superclassName + " {\n");
90      builder.append("protected void addEventHandlers() {\n");
91      var props = new Properties();
92      props.load(Files.newBufferedReader(
93         Path.of(superclassName.replace(".", "/")).getParent().resolve("action.properties"),
94         StandardCharsets.UTF_8));
95      for (Map.Entry<Object, Object> e : props.entrySet())
96      {
97         var beanName = (String) e.getKey();
98         var eventCode = (String) e.getValue();
99         builder.append(beanName + ".addActionListener(event -> {\n");
100        builder.append(eventCode);
101        builder.append("\n} );\n");
102     }
103     builder.append("} }\n");
104     return new StringSource("x.Frame", builder.toString());
```

```
105    }
106 }
```

Listing 8.4

buttons2/ButtonFrame.java

Click here to view code image

```
 1 package buttons2;
 2 import javax.swing.*;
 3
 4 /**
 5  * A frame with a button panel.
 6  * @version 1.00 2007-11-02
 7  * @author Cay Horstmann
 8  */
 9 public abstract class ButtonFrame extends JFrame
10 {
11    public static final int DEFAULT_WIDTH = 300;
12    public static final int DEFAULT_HEIGHT = 200;
13
14    protected JPanel panel;
15    protected JButton yellowButton;
16    protected JButton blueButton;
17    protected JButton redButton;
18
19    protected abstract void addEventHandlers();
20
21    public ButtonFrame()
```

```
22    {
23       setSize(DEFAULT_WIDTH, DEFAULT_HEIGHT);
24
25       panel = new JPanel();
26       add(panel);
27
28       yellowButton = new JButton("Yellow");
29       blueButton = new JButton("Blue");
30       redButton = new JButton("Red");
31
32       panel.add(yellowButton);
33       panel.add(blueButton);
34       panel.add(redButton);
35
36       addEventHandlers();
37    }
38 }
```

Listing 8.5

buttons2/action.properties

Click here to view code image

```
1 yellowButton=panel.setBackground(java.awt.Color.YELLOW);
2 blueButton=panel.setBackground(java.awt.Color.BLUE);
```

javax.tools.Tool

6

- `int run(InputStream in, OutputStream out, OutputStream err, String... arguments)`

  runs the tool with the given input, output, and error streams and the given arguments. Returns

  `0`

  for success, a nonzero value for failure.

javax.tools.JavaCompiler

6

- `StandardJavaFileManager getStandardFileManager(DiagnosticListener<? super JavaFileObject> diagnosticListener, Locale locale, Charset charset)`

  gets the standard file manager for this compiler. You can supply

  `null`

  for default error reporting, locale, and character set.
- `JavaCompiler.CompilationTask getTask(Writer out, JavaFileManager fileManager, DiagnosticListener<? super JavaFileObject> diagnosticListener, Iterable<String> options, Iterable<String> classesForAnnotationProcessing, Iterable<? extends JavaFileObject> sourceFiles)`

  gets a compilation task that, when called, will compile the given source files. See the discussion in the preceding section for details.

javax.tools.StandardJavaFileManager

6

- `Iterable<? extends JavaFileObject> getJavaFileObjectsFromStrings(Iterable<String> fileNames)`
- `Iterable<? extends JavaFileObject> getJavaFileObjectsFromPaths(Collection<? extends Path> paths)`

  13
- `Iterable<? extends JavaFileObject> getJavaFileObjectsFromFiles(Iterable<? extends File> files)`

  translate a sequence of file names or files into a sequence of

  `JavaFileObject`

  instances.

---

javax.tools.JavaCompiler.CompilationTask

6

- `Boolean call()`

  performs the compilation task.

---

javax.tools.DiagnosticCollector<S>

6

- `DiagnosticCollector()`

  constructs an empty collector.
- `List<Diagnostic<? extends S>> getDiagnostics()`

gets the collected diagnostics.

---

javax.tools.Diagnostic<S>

6

- `S getSource()`
  gets the source object associated with this diagnostic.
- `Diagnostic.Kind getKind()`
  gets the type of this diagnostic—one of
  `ERROR`
  ,
  `WARNING`
  ,
  `MANDATORY_WARNING`
  ,
  `NOTE`
  , or
  `OTHER`
  .
- `String getMessage(Locale locale)`
  gets the message describing the issue raised in this diagnostic. Pass
  `null`
  for the default locale.
- `long getLineNumber()`

- `long getColumnNumber()`

  get the position of the issue raised in this diagnostic.

`javax.tools.SimpleJavaFileObject`

6

- `CharSequence getCharContent(boolean ignoreEncodingErrors)`

  Override this method for a file object that represents a source file and produces the source code.
- `OutputStream openOutputStream()`

  Override this method for a file object that represents a class file and produces a stream to which the bytecodes can be written.

`javax.tools.ForwardingJavaFileManager<M extends JavaFileManager>`

6

- `protected ForwardingJavaFileManager(M fileManager)`

  constructs a

  `JavaFileManager`

  that delegates all calls to the given file manager.
- `FileObject getFileForOutput(JavaFileManager.Location location, String className, JavaFileObject.Kind kind, FileObject sibling)`

  Intercept this call if you want to substitute a file object for writing class files;

  `kind`

  is one of

`SOURCE`

,

`CLASS`

,

`HTML`

, or

`OTHER`

.

8.3 Using Annotations

Annotations are tags that you insert into your source code so that some tool can process them. The tools can operate on the source level, or they can process class files into which the compiler has placed annotations.

Annotations do not change the way in which your programs are compiled. The Java compiler generates the same virtual machine instructions with or without the annotations.

To benefit from annotations, you need to select a *processing tool*. Use annotations that your processing tool understands, then apply the processing tool to your code.

There is a wide range of uses for annotations, and that generality can be confusing at first. Here are some uses for annotations:

- Automatic generation of auxiliary files, such as deployment descriptors or bean information classes
- Automatic generation of code for testing, logging, transaction semantics, and so on

8.3.1 An Introduction into Annotations

I'll start our discussion of annotations with the basic concepts and put them to use in a concrete example, in which I will mark methods as event listeners for AWT components and show you an annotation processor that analyzes the annotations and hooks up the listeners. I'll then discuss the syntax rules in detail and finish the chapter with two advanced examples of annotation processing. One of them processes source-level annotations, the other uses the Apache Bytecode Engineering Library to process class files, injecting additional bytecodes into annotated methods.

Here is an example of a simple annotation:

Click here to view code image

```
public class MyClass
{
  . . .
  @Test public void checkRandomInsertions()
}
```

The annotation

```
@Test
```

annotates the

```
checkRandomInsertions
```

method.

In Java, an annotation is used like a *modifier* and is placed before the annotated item *without a semicolon*. (A modifier is a keyword such as

`public`

or

`static`

.) The name of each annotation is preceded by an

`@`

symbol, similar to Javadoc comments. However, Javadoc comments occur inside the

`/** . . . */`

delimiters, whereas annotations are part of the code.

By itself, the

`@Test`

annotation does not do anything. It needs a tool to be useful. For example, the JUnit testing tool (available at

http://junit.org

) calls all methods that are labeled

`@Test`

when testing a class. Another tool might remove all test methods from a class file so they are not shipped with the program after it has been tested.

Annotations can be defined to have *elements*, such as

```
@Test(timeout="10000")
```

These elements can be processed by the tools that read the annotations. Other forms of elements are possible; we'll discuss them later in this chapter.

Besides methods, you can annotate classes, fields, and local variables—an annotation can be anywhere you could put a modifier such as

`public`

or

`static`

. In addition, as you will see in **Section 8.4**, "**Annotation Syntax**," on p. **475**, you can annotate packages, parameter variables, type parameters, and type uses.

Each annotation must be defined by an *annotation interface*. The methods of the interface correspond to the elements of the annotation. For example, the JUnit

`Test`

annotation is defined by the following interface:

Click here to view code image

```
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface Test
{
  long timeout() default 0L;
  . . .
}
```

The

`@interface`

declaration creates an actual Java interface. Tools that process annotations receive objects that implement the annotation interface. A tool would call the

`timeout`

method to retrieve the

`timeout`

element of a particular

`Test`

annotation.

The

`Target`

and

`Retention`

annotations are *meta-annotations*. They annotate the

`Test`

annotation, marking it as an annotation that can be applied to methods only and is retained when the class file is loaded into the virtual machine. We'll discuss these in detail in **Section 8.5.2**, "**Meta-Annotations**," on p. **484**.

You have now seen the basic concepts of program metadata and annotations. In the next section, we'll walk through a concrete example of annotation processing.



Note

For a compelling use of annotations, check out

`JCommander`

(

http://jcommander.org

) and picocli (

http://picocli.info

). These libraries use annotations for the processing of command-line parameters.

8.3.2 An Example: Annotating Event Handlers

One of the more boring tasks in user interface programming is the wiring of listeners to event sources. Many listeners are of the form

Click here to view code image

```
myButton.addActionListener(() -> doSomething());
```

In this section, we'll design an annotation to reverse the wiring. The annotation, defined in Listing 8.8, is used as follows:

Click here to view code image

```
@ActionListenerFor(source="myButton") void doSomething() { . . . }
```

The programmer no longer has to make calls to

```
addActionListener
```

. Instead, each method is tagged with an annotation. Listing 8.7 shows the

`ButtonFrame`

class from Volume I, Chapter 10, reimplemented with these annotations.

We also need to define an annotation interface. The code is in Listing 8.8.

Of course, the annotations don't do anything by themselves. They sit in the source file. The compiler places them in the class file, and the virtual machine loads them. We now need a mechanism to analyze them and install action listeners. That is the job of the

`ActionListenerInstaller`

class. The

`ButtonFrame`

constructor calls

Click here to view code image

```
ActionListenerInstaller.processAnnotations(this);
```

The static

`processAnnotations`

method enumerates all methods of the object it received. For each method, it gets the

`ActionListenerFor`

annotation object and processes it.

Click here to view code image

```
Class<?> cl = obj.getClass();
for (Method m : cl.getDeclaredMethods())
{
  ActionListenerFor a = m.getAnnotation(ActionListenerFor.class);
  if (a != null) . . .
}
```

Here, we use the `getAnnotation` method defined in the `AnnotatedElement` interface. The classes `Method`, `Constructor`, `Field`, `Class`, and `Package` implement this interface.

The name of the source field is stored in the annotation object. We retrieve it by calling the

`source`

method, and then look up the matching field.

Click here to view code image

```
String fieldName = a.source();
Field f = cl.getDeclaredField(fieldName);
```

This shows a limitation of our annotation. The source element must be the name of a field. It cannot be a local variable.

The remainder of the code is rather technical. For each annotated method, we construct a proxy object, implementing the `ActionListener` interface, with an `actionPerformed` method that calls the annotated method. (For more information about proxies, see Volume I, Chapter 6.) The details are not important. The key observation is that the functionality of the annotations was established by the `processAnnotations` method.

Figure 8.1 shows how annotations are handled in this example.

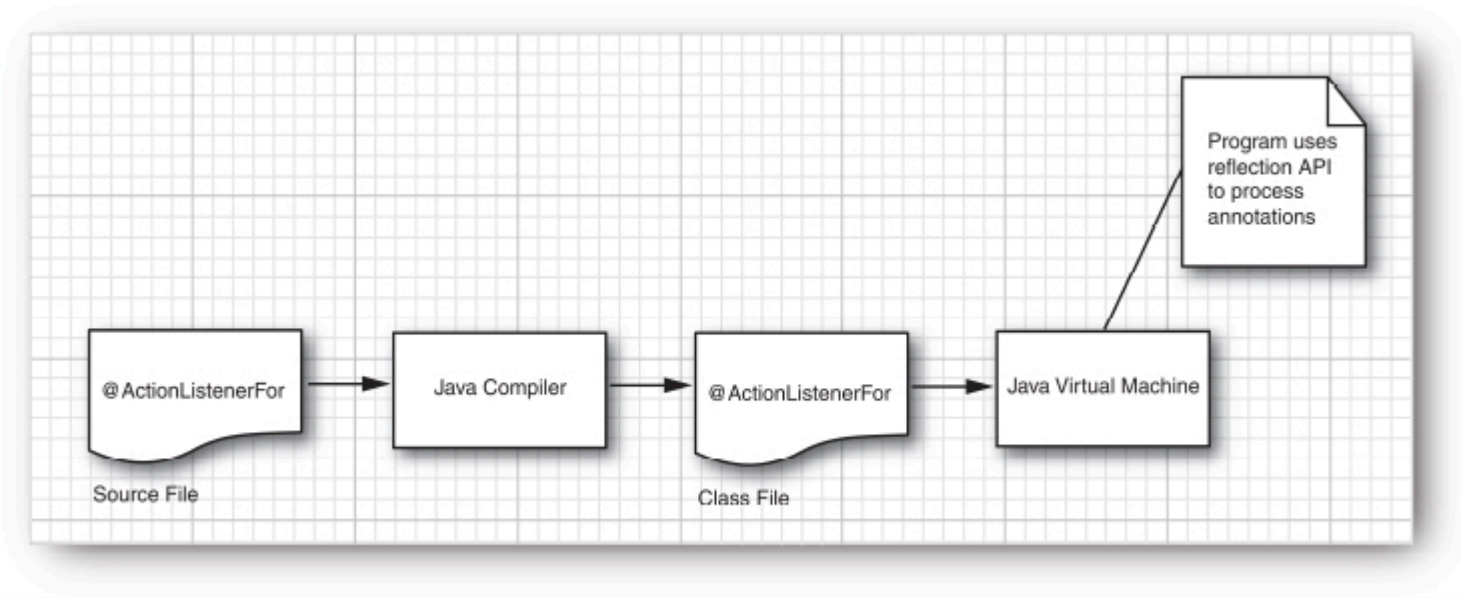


Figure 8.1 Processing annotations at runtime

In this example, the annotations were processed at runtime. It is also possible to process them at the source level: A source code generator would then produce the code for adding the listeners. Alternatively, the annotations can be processed at the bytecode level: A bytecode editor could inject the calls to

```
addActionListener
```

into the frame constructor. This sounds complex, but libraries are available to make this task relatively straightforward. You can see an example in **Section 8.7**, "**Bytecode Engineering**," on p. **493**.

Our example was not intended as a serious tool for user interface programmers. A utility method for adding a listener could be just as convenient for the programmer as the annotation. (In fact, the

```
java.beans.EventHandler
```

class tries to do just that. You could make the class truly useful by supplying a method that adds the event handler instead of just constructing it.)

However, this example shows the mechanics of annotating a program and of analyzing the annotations. Having seen a concrete example, you are now more prepared (I hope) for the following sections that describe the annotation syntax in complete detail.

Listing 8.6

runtimeAnnotations/ActionListenerInstaller.java

Click here to view code image

```
 1 package runtimeAnnotations;
 2
 3 import java.awt.event.*;
 4 import java.lang.reflect.*;
 5
 6 /**
 7  * @version 1.00 2004-08-17
 8  * @author Cay Horstmann
 9  */
10 public class ActionListenerInstaller
11 {
12   /**
13    * Processes all ActionListenerFor annotations in the given object.
14    * @param obj an object whose methods may have ActionListenerFor annotations
15    */
16   public static void processAnnotations(Object obj)
17   {
18     try
19     {
```

```
      Class<?> cl = obj.getClass();
      for (Method m : cl.getDeclaredMethods())
      {
         ActionListenerFor a = m.getAnnotation(ActionListenerFor.class);
         if (a != null)
         {
            Field f = cl.getDeclaredField(a.source());
            f.setAccessible(true);
            addListener(f.get(obj), obj, m);
         }
       }
    }
    catch (ReflectiveOperationException e)
    {
      e.printStackTrace();
    }
}

/**
 * Adds an action listener that calls a given method.
 * @param source the event source to which an action listener is added
 * @param param the implicit parameter of the method that the listener calls
 * @param m the method that the listener calls
 */
public static void addListener(Object source, final Object param, final Method m)
      throws ReflectiveOperationException
{
   var handler = new InvocationHandler()
      {
         public Object invoke(Object proxy, Method mm, Object[] args) throws Throwable
         {
           return m.invoke(param);
         }
```

```
53         };
54
55      Object listener = Proxy.newProxyInstance(null,
56         new Class[] { java.awt.event.ActionListener.class }, handler);
57      Method adder = source.getClass().getMethod("addActionListener", ActionListener.class);
58      adder.invoke(source, listener);
59   }
60 }
```

Listing 8.7

buttons3/ButtonFrame.java

Click here to view code image

```
 1 package buttons3;
 2
 3 import java.awt.*;
 4 import javax.swing.*;
 5 import runtimeAnnotations.*;
 6
 7 /**
 8  * A frame with a button panel.
 9  * @version 1.00 2004-08-17
10  * @author Cay Horstmann
11  */
12 public class ButtonFrame extends JFrame
13 {
14    private static final int DEFAULT_WIDTH = 300;
15    private static final int DEFAULT_HEIGHT = 200;
```

```
16
17    private JPanel panel;
18    private JButton yellowButton;
19    private JButton blueButton;
20    private JButton redButton;
21
22    public ButtonFrame()
23    {
24       setSize(DEFAULT_WIDTH, DEFAULT_HEIGHT);
25
26       panel = new JPanel();
27       add(panel);
28
29       yellowButton = new JButton("Yellow");
30       blueButton = new JButton("Blue");
31       redButton = new JButton("Red");
32
33       panel.add(yellowButton);
34       panel.add(blueButton);
35       panel.add(redButton);
36
37       ActionListenerInstaller.processAnnotations(this);
38    }
39
40    @ActionListenerFor(source = "yellowButton")
41    public void yellowBackground()
42    {
43       panel.setBackground(Color.YELLOW);
44    }
45
46    @ActionListenerFor(source = "blueButton")
47    public void blueBackground()
48    {
```

```
49       panel.setBackground(Color.BLUE);
50    }
51
52    @ActionListenerFor(source = "redButton")
53    public void redBackground()
54    {
55       panel.setBackground(Color.RED);
56    }
57 }
```

Listing 8.8

runtimeAnnotations/ActionListenerFor.java

Click here to view code image

```
 1 package runtimeAnnotations;
 2
 3 import java.lang.annotation.*;
 4
 5 /**
 6  * @version 1.00 2004-08-17
 7  * @author Cay Horstmann
 8  */
 9 @Target(ElementType.METHOD)
10 @Retention(RetentionPolicy.RUNTIME)
11 public @interface ActionListenerFor
12 {
```

```
13    String source();
14 }
```

java.lang.reflect.AnnotatedElement

5.0

- `boolean isAnnotationPresent(Class<? extends Annotation> annotationType)`

  returns

  `true`

  if this item has an annotation of the given type.
- `<T extends Annotation> T getAnnotation(Class<T> annotationType)`

  gets the annotation of the given type, or

  `null`

  if this item has no such annotation.
- `<T extends Annotation> T[] getAnnotationsByType(Class<T> annotationType)`

  8

  gets all annotations of a repeatable annotation type (see **Section 8.5.2**, "**Meta-Annotations**," on p. **484**), or an array of length 0.
- `Annotation[] getAnnotations()`

  gets all annotations present for this item, including inherited annotations. If no annotations are present, an array of length 0 is returned.
- `Annotation[] getDeclaredAnnotations()`

  gets all annotations declared for this item, excluding inherited annotations. If no annotations are present, an array of length 0 is returned.

8.4 Annotation Syntax

In the following sections, we cover everything you need to know about the annotation syntax.

8.4.1 Annotation Interfaces

An annotation is defined by an annotation interface:

Click here to view code image

```
modifiers @interface AnnotationName
{
    elementDeclaration1
    elementDeclaration2
    . . .
}
```

Each element declaration has the form

```
type elementName();
```

or

Click here to view code image

```
type elementName() default value;
```

For example, the following annotation has two elements,

`assignedTo`

and

`severity`

:

Click here to view code image

```
public @interface BugReport
{
  String assignedTo() default "[none]";
  int severity();
}
```

All annotation interfaces implicitly extend the

`java.lang.annotation.Annotation`

interface. That interface is a regular interface, *not* an annotation interface. See the API notes at the end of this section for the methods provided by this interface. You cannot extend an annotation interface—in other words, all annotation interfaces directly extend

`java.lang.annotation.Annotation`

. You never supply classes that implement annotation interfaces.

The methods of an annotation interface have no parameters and no `throws` clauses. They cannot be `default` or `static` methods, and they cannot have type parameters.

The type of an annotation element is one of the following:

- A primitive type (`int`, `short`, `long`, `byte`, `char`, `double`, `float`, or `boolean`)

)

- `String`
- `Class`
  (with an optional type parameter such as
  `Class<? extends MyClass>`
)
- An
  `enum`
  type
- An annotation type
- An array of the preceding types (an array of arrays is not a legal element type)

Here are examples of valid element declarations:

Click here to view code image

```
public @interface BugReport
{
  enum Status { UNCONFIRMED, CONFIRMED, FIXED, NOTABUG };
  boolean showStopper() default false;
  String assignedTo() default "[none]";
  Class<?> testCase() default Void.class;
  Status status() default Status.UNCONFIRMED;
  Reference ref() default @Reference(); // an annotation type
  String[] reportedBy();
}
```

java.lang.annotation.Annotation

5.0

- `Class<? extends Annotation> annotationType()`

  returns the

  `Class`

  object that represents the annotation interface of this annotation object. Note that calling

  `getClass`

  on an annotation object would return the actual class, not the interface.
- `boolean equals(Object other)`

  returns

  `true`

  if

  `other`

  is an object that implements the same annotation interface as this annotation object and if all elements of this object and

  `other`

  are equal.
- `int hashCode()`

  returns a hash code, compatible with the

  `equals`

  method, derived from the name of the annotation interface and the element values.
- `String toString()`

  returns a string representation that contains the annotation interface name and the element values, for example

  `@BugReport(assignedTo=[none], severity=0)`

.

8.4.2 Annotations

Each annotation has the format

Click here to view code image

```
@AnnotationName(elementName1=value1, elementName2=value2, . . .)
```

For example,

Click here to view code image

```
@BugReport(assignedTo="Harry", severity=10)
```

The order of the elements does not matter. The annotation

Click here to view code image

```
@BugReport(severity=10, assignedTo="Harry")
```

is identical to the preceding one.

The default value of the declaration is used if an element value is not specified. For example, consider the annotation

```
@BugReport(severity=10)
```

The value of the `assignedTo` element is the string `"[none]"`.



Caution

Defaults are not stored with the annotation; instead, they are dynamically computed. For example, if you change the default for the `assignedTo` element to `"[]"` and recompile the `BugReport` interface, the annotation `@BugReport(severity=10)`

will use the new default, even in class files that have not been recompiled after the default changed.

Two special shortcuts can simplify annotations.

If no elements are specified, either because the annotation doesn't have any or because all of them use the default value, you don't need to use parentheses. For example,

```
@BugReport
```

is the same as

Click here to view code image

```
@BugReport(assignedTo="[none]", severity=0)
```

Such an annotation is called a *marker annotation*.

The other shortcut is the *single-value annotation*. If an element has the special name

```
value
```

and no other element is specified, you can omit the element name and the

```
=
```

symbol. For example, had we defined the

ActionListenerFor

annotation interface of the preceding section as

Click here to view code image

```
public @interface ActionListenerFor
{
  String value();
}
```

then the annotations could be written as

Click here to view code image

```
@ActionListenerFor("yellowButton")
```

instead of

Click here to view code image

```
@ActionListenerFor(value="yellowButton")
```

An item can have multiple annotations:

Click here to view code image

```
@Test
@BugReport(showStopper=true, reportedBy="Joe")
public void checkRandomInsertions()
```

If the author of an annotation declared it to be repeatable, you can repeat the same annotation multiple times:

Click here to view code image

```
@BugReport(showStopper=true, reportedBy="Joe")
@BugReport(reportedBy={"Harry", "Carl"})
public void checkRandomInsertions()
```



Note

Since annotations are evaluated by the compiler, all element values must be compile-time constants. For example,

Click here to view code image

```
@BugReport(showStopper=true, assignedTo="Harry", testCase=MyTestCase.class,
  status=BugReport.Status.CONFIRMED, . . .)
```



Caution

An annotation element can never be set to `null`. Not even a `default` of `null` is permissible. This can be rather inconvenient in practice. You will need to find other defaults, such as `""` or `Void.class`.

If an element value is an array, enclose its values in braces:

Click here to view code image

```
@BugReport(. . ., reportedBy={"Harry", "Carl"})
```

You can omit the braces if the element has a single value:

Click here to view code image

```
@BugReport(. . ., reportedBy="Joe") // OK, same as {"Joe"}
```

Since an annotation element can be another annotation, you can build arbitrarily complex annotations. For example,

Click here to view code image

```
@BugReport(ref=@Reference(id="3352627"), . . .)
```



Note

It is an error to introduce circular dependencies in annotations. For example,

```
BugReport
```

has an element of the annotation type

```
Reference
```

, therefore

`Reference`

cannot have an element of type

`BugReport`

.

8.4.3 Annotating Declarations

There are many places where annotations can occur. They fall into two categories: *declarations* and *type uses*. Declaration annotations can appear at the declarations of

- Packages
- Classes (including `enum`)
- Interfaces (including annotation interfaces)
- Methods
- Constructors
- Instance fields (including `enum` constants)
- Local variables
- Parameter variables
- Type parameters

For classes and interfaces, put the annotations before the

class

or

interface

keyword:

Click here to view code image

```
@Entity public class User { . . . }
```

For variables, put them before the type:

Click here to view code image

```
@SuppressWarnings("unchecked") List<User> users = . . .;
public User getUser(@Param("id") String userId)
```

A type parameter in a generic class or method can be annotated like this:

Click here to view code image

```
public class Cache<@Immutable V> { . . . }
```

A package is annotated in a file

```
package-info.java
```

that contains only the package statement preceded by annotations.

Click here to view code image

```
/**
   Package-level Javadoc
*/
@GPL(version="3")
package com.horstmann.corejava;
import org.gnu.GPL;
```



Note

Annotations for local variables can only be processed at the source level. Class files do not describe local variables. Therefore, all local variable annotations are discarded when a class is compiled. Similarly, annotations for packages are not retained beyond the source level.

8.4.4 Annotating Type Uses

A declaration annotation provides some information about the item being declared. For example, in the declaration

Click here to view code image

```
public User getUser(@NonNull String userId)
```

it is asserted that the

`userId`

parameter is not

`null`

.



Note

The

`@NonNull`

annotation is a part of the Checker Framework (

https://checkerframework.org

). With that framework, you can include assertions in your program—for example, that a parameter is non-null or that a

`String`

contains a regular expression. A static analysis tool then checks whether the assertions are valid in a given body of source code.

Now, suppose we have a parameter of type

`List<String>`

, and we want to express that all of the strings are non-null. That is where type use annotations come in. Place the annotation before the type argument:

```
List<@NonNull String>
```

.

Type use annotations can appear in the following places:

- With generic type arguments:

  `List<@NonNull String>`

  ,

  `Comparator.<@NonNull String> reverseOrder()`

  .
- In any position of an array:

  `@NonNull String[][] words`

  (

  `words[i][j]`

  is not

  `null`

  ),

  `String @NonNull [][] words`

  (

  `words`

  is not

  `null`

  ),

  `String[] @NonNull [] words`

(
`words[i]`
is not
`null`
).

- With superclasses and implemented interfaces:
  `class Warning extends @Localized Message`
  .
- With constructor invocations:
  `new @Localized String(. . .)`
  .
- With casts and
  `instanceof`
  checks:
  `(@Localized String) text`
  ,
  `if (text instanceof @Localized String)`
  . (The annotations are only for use by external tools. They have no effect on the behavior of a cast or an
  `instanceof`
  check.)
- With exception specifications:
  `public String read() throws @Localized IOException`
  .
- With wildcards and type bounds:
  `List<@Localized ? extends Message>`
  ,

`List<? extends @Localized Message>`

.

- With method and constructor references:

  `@Localized Message::getText`

  .

There are a few type positions that cannot be annotated:

Click here to view code image

```
@NonNull String.class // ERROR: Cannot annotate class literal
import java.lang.@NonNull String; // ERROR: Cannot annotate import
```

You can place annotations before or after other modifiers such as `private` and `static`. It is customary (but not required) to put type use annotations after other modifiers, and declaration annotations before other modifiers. For example,

Click here to view code image

```
private @NonNull String text; // Annotates the type use
@Id private String userId; // Annotates the variable
```

When annotating a record component, the annotation also applies to the generated field, getter method, and constructor parameter.



Note

An annotation author needs to specify where a particular annotation can appear. If an annotation is permissible both for a variable and a type use, and it is used in a variable declaration, then both the variable and the type use are annotated. For example, consider

Click here to view code image

```
public User getUser(@NonNull String userId)
```

If `@NonNull` can apply both to parameters and to type uses, the `userId` parameter is annotated, and the parameter type is `@NonNull String`.

8.4.5 Annotating

this

Suppose you want to annotate parameters that are not being mutated by a method.

Click here to view code image

```
public class Point
{
  public boolean equals(@ReadOnly Object other) { . . . }
}
```

Then a tool that processes this annotation would, upon seeing a call

```
p.equals(q)
```

reason that

q

has not been changed.

But what about

p

?

When the method is called, the

`this`

variable is bound to

`p`

. But

`this`

is never declared, so you cannot annotate it.

Actually, you can declare it, with a rarely used syntax variant, just so that you can add an annotation:

Click here to view code image

```
public class Point
{
  public boolean equals(@ReadOnly Point this, @ReadOnly Object other) { . . . }
}
```

The first parameter is called the *receiver parameter*. It must be named

`this`

. Its type is the class that is being constructed.



Note

You can provide a receiver parameter only for methods, not for constructors. Conceptually, the

`this`

reference in a constructor is not an object of the given type until the constructor has completed. Instead, an annotation placed on the constructor describes a property of the constructed object.

A different hidden parameter is passed to the constructor of an inner class, namely the reference to the enclosing class object. You can make that parameter explicit as well:

Click here to view code image

```
public class Sequence
{
  private int from;
  private int to;

  class Iterator implements java.util.Iterator<Integer>
  {
    private int current;
    public Iterator(@ReadOnly Sequence Sequence.this)
    {
      this.current = Sequence.this.from;
    }
    . . .
  }
  . . .
}
```

The parameter must be named just like when you refer to it,

EnclosingClass.this

, and its type is the enclosing class.

8.5 Standard Annotations

A number of annotation interfaces are defined in the `java.lang`, `java.lang.annotation`, and `javax.annotation` packages. Four of them are meta-annotations that describe the behavior of annotation interfaces. The others are regular annotations that you can use to annotate items in your source code. **Table 8.2** shows these annotations. We'll discuss them in detail in the following two sections.

Table 8.2 The Standard Annotations

Annotation Interface	Applicable To	Purpose
`Deprecated`	All	Marks item as deprecated.
`SuppressWarnings`	All but packages and annotations	Suppresses warnings of the given type.
`SafeVarargs`	Methods and constructors	Asserts that the varargs parameter is safe to use.

Annotation Interface	Applicable To	Purpose
Override	Methods	Checks that this method overrides a superclass method.
Serial	Methods	Checks that this method is a correct serialization method.
FunctionalInterface	Interfaces	Marks an interface as functional (with a single abstract method).
Generated	All	Marks an item as source code that has been generated by a tool.
Target	Annotations	Specifies the items to which this annotation can be applied.
Retention	Annotations	Specifies how long this annotation is retained.
Documented	Annotations	Specifies that this annotation should be included in the documentation of annotated items.
Inherited	Annotations	Specifies that this annotation, when applied to a class, is automatically inherited by its subclasses.
Repeatable	Annotations	Specifies that this annotation can be applied multiple times to the same item.

8.5.1 Annotations for Compilation

The `@Deprecated` annotation can be attached to any items whose use is no longer encouraged. The compiler will warn when you use a deprecated item. This annotation has the same role as the `@deprecated` Javadoc tag. However, the annotation persists until runtime.



Note

The `jdeprscan` utility that is part of the JDK can scan a set of JAR files for deprecated elements.

The `@SuppressWarnings` annotation tells the compiler to suppress warnings of a particular type, for example:

Click here to view code image

```
@SuppressWarnings("unchecked")
```

The

`@Override`

annotation applies only to methods. The compiler checks that a method with this annotation really overrides a method from the superclass. For example, if you declare

Click here to view code image

```
public MyClass
{
  @Override public boolean equals(MyClass other);
  . . .
}
```

then the compiler will report an error. After all, the `equals` method does *not* override the `equals` method of the `Object` class because that method has a parameter of type `Object`, not `MyClass`.

The

`@Generated`

annotation is intended for use by code generator tools. Any generated source code can be annotated to differentiate it from programmer-created code. For example, a code editor can hide the generated code, or a code generator can remove older versions of generated code. Each annotation must contain a unique identifier for the code generator. A date string (in ISO 8601 format) and a comment string are optional. For example,

Click here to view code image

```
@Generated("com.horstmann.beanproperty", "2008-01-04T12:08:56.235-0700");
```

8.5.2 Meta-Annotations

The

`@Target`

meta-annotation is applied to an annotation, restricting the items to which the annotation applies. For example,

Click here to view code image

```
@Target({ElementType.TYPE, ElementType.METHOD})
public @interface BugReport
```

Table 8.3 shows all possible values. They belong to the enumerated type

`ElementType`

. You can specify any number of element types, enclosed in braces.

Table 8.3 Element Types for the `@Target` Annotation

Element Type	Annotation Applies To
`ANNOTATION_TYPE`	Annotation type declarations
`PACKAGE`	Packages
`TYPE`	Classes (including `enum`) and interfaces (including annotation types)
`METHOD`	Methods
`CONSTRUCTOR`	Constructors
`FIELD`	Fields (including `enum` constants)
`PARAMETER`	Method or constructor parameters

Element Type	Annotation Applies To
LOCAL_VARIABLE	Local variables
TYPE_PARAMETER	Type parameters
TYPE_USE	Uses of a type

An annotation without an `@Target` restriction can be applied to any item. The compiler checks that you apply an annotation only to a permitted item. For example, if you apply `@BugReport` to a field, a compile-time error results.

The `@Retention` meta-annotation specifies how long an annotation is retained. You can specify at most one of the values in **Table 8.4**. The default is `RetentionPolicy.CLASS`.

Table 8.4 Retention Policies for the `@Retention` Annotation

Retention Policy	Description
`SOURCE`	Annotations are not included in class files.
`CLASS`	Annotations are included in class files, but the virtual machine need not load them.
`RUNTIME`	Annotations are included in class files and loaded by the virtual machine. They are available through the reflection API.

In **Listing 8.8** on p. 474, the `@ActionListenerFor` annotation was declared with `RetentionPolicy.RUNTIME` because we used reflection to process annotations. In the following two sections, you will see examples of processing annotations at the source and class file levels.

The `@Documented` meta-annotation gives a hint to documentation tools such as Javadoc. Documented annotations should be treated just like other modifiers, such as `protected` or `static`

, for documentation purposes. The use of other annotations is not included in the documentation. For example, suppose we declare

```
@ActionListenerFor
```

as a documented annotation:

Click here to view code image

```
@Documented
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface ActionListenerFor
```

Now the documentation of each annotated method contains the annotation, as shown in Figure 8.2.

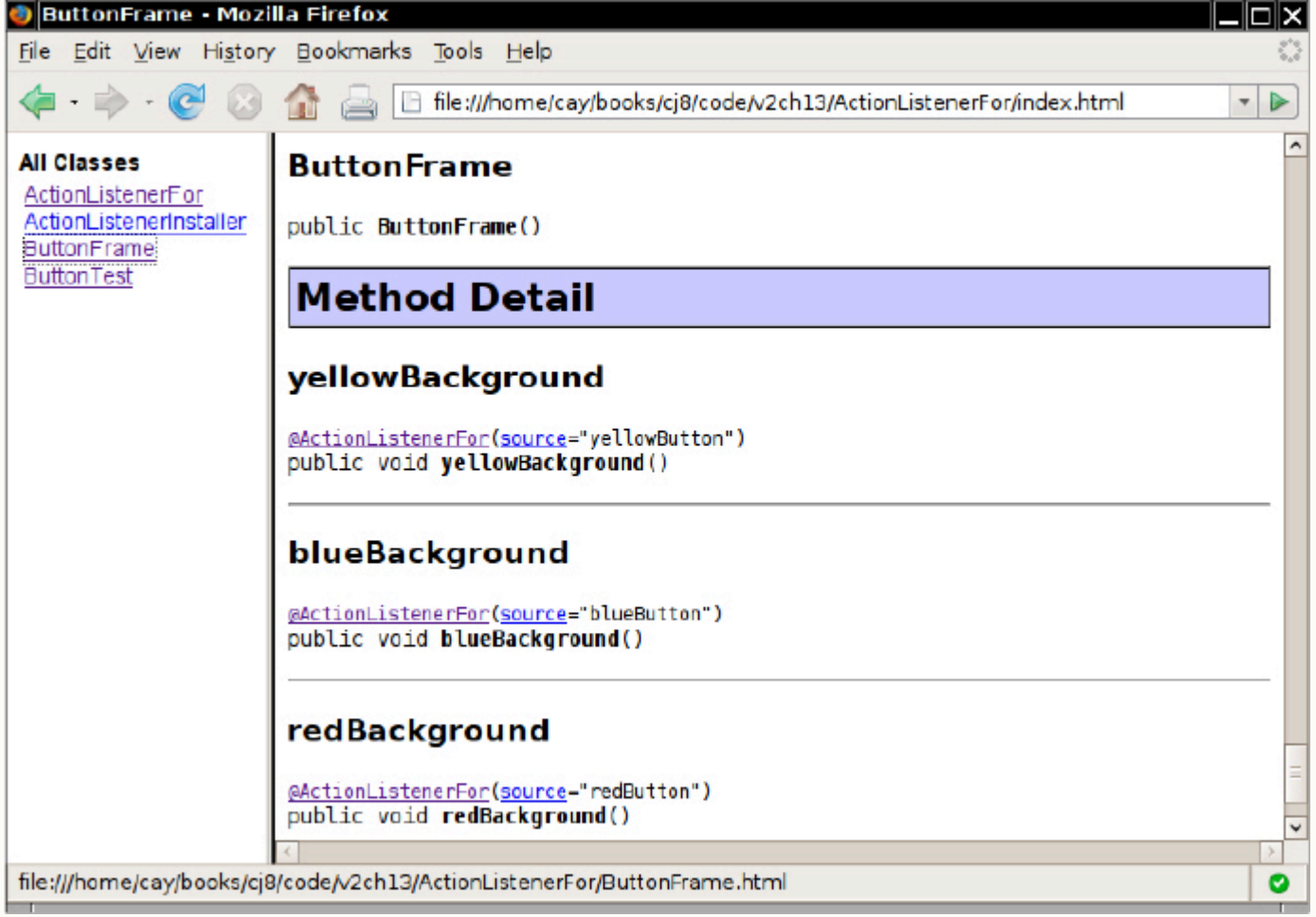


Figure 8.2 Documented annotations

If an annotation is transient (such as

```
@BugReport
```

), you should probably not document its use.

 **Note**

It is legal to apply an annotation to itself. For example, the `@Documented` annotation is itself annotated as `@Documented`. Therefore, the Javadoc documentation for annotations shows whether they are documented.

The `@Inherited` meta-annotation applies only to annotations for classes. When a class has an inherited annotation, then all of its subclasses automatically have the same annotation. This makes it easy to create annotations that work as marker interfaces, such as `Serializable`.

In fact, an annotation `@Serializable` would be more appropriate than the `Serializable` marker interface with no methods. A class is serializable because there is runtime support for reading and writing its fields, not because of any principles of object-oriented design. An annotation describes this fact better than does interface inheritance. Of course, the `Serializable` interface was created in JDK 1.1, long before annotations existed.

Suppose you define an inherited annotation

`@Persistent`

to indicate that objects of a class can be saved in a database. Then the subclasses of persistent classes are automatically annotated as persistent.

Click here to view code image

```
@Inherited @interface Persistent { }
@Persistent class Employee { . . . }
class Manager extends Employee { . . . } // also @Persistent
```

When the persistence mechanism searches for objects to store in the database, it will detect both

`Employee`

and

`Manager`

objects.

As of Java 8, it is legal to apply the same annotation type multiple times to an item. For backward compatibility, the implementor of a repeatable annotation needs to provide a *container annotation* that holds the repeated annotations in an array.

Here is how to define the

`@TestCase`

annotation and its container:

```
@Repeatable(TestCases.class)
@interface TestCase
{
  String params();
  String expected();
}

@interface TestCases
{
  TestCase[] value();
}
```

Whenever the user supplies two or more `@TestCase` annotations, they are automatically wrapped into a `@TestCases` annotation.



Caution

You have to be careful when processing repeatable annotations. If you call `getAnnotation` to look up a repeatable annotation, and the annotation was actually repeated, then you get `null`. That is because the repeated annotations were wrapped into the container annotation.

In that case, you should call

`getAnnotationsByType`

. That call "looks through" the container and gives you an array of the repeated annotations. If there was just one annotation, you get it in an array of length 1. With this method, you don't have to worry about the container annotation.

8.6 Source-Level Annotation Processing

In the preceding section, you saw how to analyze annotations in a running program. Another use for annotation is the automatic processing of source files to produce more source code, configuration files, scripts, or whatever else one might want to generate.

8.6.1 Annotation Processors

Annotation processing is integrated into the Java compiler. During compilation, you can *invoke annotation processors* by running

Click here to view code image

```
javac -processor ProcessorClassName1,ProcessorClassName2,. . . sourceFiles
```

The compiler locates the annotations of the source files. Each annotation processor is executed in turn and given the annotations in which it expressed an interest. If an annotation processor creates a new source file, the process is repeated. Once a processing round yields no further source files, all source files are compiled.



Note

An annotation processor can only generate new source files. It cannot modify an existing source file.

An annotation processor implements the

`Processor`

interface, generally by extending the

`AbstractProcessor`

class. You need to specify which annotations your processor supports. In our case:

Click here to view code image

```
@SupportedAnnotationTypes("com.horstmann.annotations.ToString")
@SupportedSourceVersion(SourceVersion.RELEASE_8)
public class ToStringAnnotationProcessor extends AbstractProcessor
{
  public boolean process(Set<? extends TypeElement> annotations,
      RoundEnvironment currentRound)
  {
    ...
  }
}
```

A processor can claim specific annotation types, wildcards such as

`"com.horstmann.*"`

(all annotations in the

`com.horstmann`

package or any subpackage), or even

`"*"`
(all annotations).

The
`process`
method is called once for each round, with the set of all annotations that were found in any files during this round, and a
`RoundEnvironment`
reference that contains information about the current processing round.

8.6.2 The Language Model API

Use the *language model* API for analyzing source-level annotations. Unlike the reflection API, which presents the virtual machine representation of classes and methods, the language model API lets you analyze a Java program according to the rules of the Java language.

The compiler produces a tree whose nodes are instances of classes that implement the
`javax.lang.model.element.Element`
interface and its subinterfaces:
`TypeElement`
,
`VariableElement`
,
`ExecutableElement`
, and so on. These are the compile-time analogs to the
`Class`
,

`Field`
/
`Parameter`
,
`Method`
/
`Constructor`
reflection classes.

I do not want to cover the API in detail, but here are the highlights that you need to know for processing annotations:

- The `RoundEnvironment` gives you a set of all elements annotated with a particular annotation. Call the method
  Click here to view code image

  ```
  Set<? extends Element> getElementsAnnotatedWith(Class<? extends Annotation> a)
  ```

- The source-level equivalent of the `AnnotateElement` interface is `AnnotatedConstruct`. Use the methods
  Click here to view code image

```
A getAnnotation(Class<A> annotationType)
A[] getAnnotationsByType(Class<A> annotationType)
```

to get the annotation or repeated annotations for a given annotation class.

- A `TypeElement` represents a class or interface. The `getEnclosedElements` method yields a list of its fields and methods.
- Calling `getSimpleName` on an `Element` or `getQualifiedName` on a `TypeElement` yields a `Name` object that can be converted to a string with `toString`.

8.6.3 Using Annotations to Generate Source Code

As an example, we will use annotations to reduce the tedium of implementing `toString` methods. We can't put these methods into the original classes—annotation processors can only produce new classes, not modify existing ones.

Therefore, we'll add all methods into a utility class `ToStrings`:

Click here to view code image

```
public class ToStrings
{
  public static String toString(Point obj)
  {
    Generated code
  }
  public static String toString(Rectangle obj)
  {
    Generated code
  }
  . . .

  public static String toString(Object obj)
  {
    return Objects.toString(obj);
```

```
  }
}
```

We don't want to use reflection, so we annotate accessor methods, not fields:

Click here to view code image

```
@ToString
public class Rectangle
{
  . . .
  @ToString(includeName=false) public Point getTopLeft() { return topLeft; }
  @ToString public int getWidth() { return width; }
  @ToString public int getHeight() { return height; }
}
```

The annotation processor should then generate the following source code:

Click here to view code image

```
public static String toString(Rectangle obj)
{
  var result = new StringBuilder();
  result.append("Rectangle");
  result.append("[");
  result.append(toString(obj.getTopLeft()));
```

```
    result.append(",");
    result.append("width=");
    result.append(toString(obj.getWidth()));
    result.append(",");
    result.append("height=");
    result.append(toString(obj.getHeight()));
    result.append("]");
    return result.toString();
  }
```

The “boilerplate” code is in gray. Here is an outline of the method that produces the

`toString`

method for a class with a given

`TypeElement`

:

Click here to view code image

```
private void writeToStringMethod(PrintWriter out, TypeElement te)
{
  String className = te.getQualifiedName().toString();
  Print method header and declaration of string builder
  ToString ann = te.getAnnotation(ToString.class);
  if (ann.includeName())
    Print code to add class name
  for (Element c : te.getEnclosedElements())
  {
    ann = c.getAnnotation(ToString.class);
```

```
    if (ann != null)
    {
      if (ann.includeName()) Print code to add field name
      Print code to append toString(obj.methodName())
    }
  }
  Print code to return string
}
```

And here is an outline of the

`process`

method of the annotation processor. It creates a source file for the helper class and writes the class header and one method for each annotated class.

Click here to view code image

```
public boolean process(Set<? extends TypeElement> annotations,
    RoundEnvironment currentRound)
{
  if (annotations.size() == 0) return true;
  try
  {
    JavaFileObject sourceFile = processingEnv.getFiler().createSourceFile(
        "com.horstmann.annotations.ToStrings");
    try (var out = new PrintWriter(sourceFile.openWriter()))
    {
      Print code for package and class
      for (Element e : currentRound.getElementsAnnotatedWith(ToString.class))
```

```
            if (e instanceof TypeElement te)
              writeToStringMethod(out, te);
          Print code for toString(Object)
        }
        catch (IOException e)
        {
          processingEnv.getMessager().printMessage(
              Kind.ERROR, ex.getMessage());
        }
      }
      return true;
    }
```

For the tedious details, check the book's companion code.

Note that the

```
process
```

method is called in subsequent rounds with an empty list of annotations. It then returns immediately so it doesn't create the source file twice.

First, compile the annotation processor, and then compile and run the test program as follows:

Click here to view code image

```
javac sourceAnnotations/ToStringAnnotationProcessor.java
javac -processor sourceAnnotations.ToStringAnnotationProcessor rect/*.java
java rect.SourceLevelAnnotationDemo
```



Tip

To see the rounds, run the

```
javac
```

command with the

```
-XprintRounds
```

flag:

Click here to view code image

```
Round 1:
  input files: {rect.Point, rect.Rectangle,
    rect.SourceLevelAnnotationDemo}
  annotations: [sourceAnnotations.ToString]
  last round: false
Round 2:
  input files: {sourceAnnotations.ToStrings}
  annotations: []
  last round: false
Round 3:
  input files: {}
  annotations: []
  last round: true
```

This example demonstrates how tools can harvest source file annotations to produce other files. The generated files don't have to be source files. Annotation processors may choose to generate XML descriptors, property files, shell scripts, HTML documentation, and so on.



Note

Some people have suggested using annotations to remove an even bigger drudgery. Wouldn't it be nice if trivial getters and setters were generated automatically? For example, the annotation

Click here to view code image

```
@Property private String title;
```

could produce the methods

Click here to view code image

```
public String getTitle() { return title; }
public void setTitle(String title) { this = title; }
```

However, those methods need to be added to the *same class*. This requires editing a source file, not just generating another file, and is beyond the capabilities of annotation processors. It would be possible to build another

tool for this purpose, but such a tool would go beyond the mission of annotations. An annotation is intended as a description *about* a code item, not a directive for adding or changing code.

8.7 Bytecode Engineering

You have seen how annotations can be processed at runtime or at the source code level. There is a third possibility: processing at the bytecode level. Unless annotations are removed at the source level, they are present in the class files. The class file format is documented (see

http://docs.oracle.com/javase/specs/jvms/se10/html

). The format is rather complex, and it would be challenging to process class files without special libraries. One such library is the ASM library, available at

https://asm.ow2.org

. Download

asm-9.2.jar

and

asm-commons-9.2.jar

and place them in a directory of your choice, called *asm* in the instructions.

8.7.1 Modifying Class Files

In this section, we use ASM to add logging messages to annotated methods. If a method is annotated with

```
@LogEntry(logger=loggerName)
```

then we add the bytecodes for the following statement at the beginning of the method:

Click here to view code image

```
Logger.getLogger(loggerName).entering(className, methodName);
```

For example, if you annotate the `hashCode` method of the `Item` class as

Click here to view code image

```
@LogEntry(logger="global") public int hashCode()
```

then a message similar to the following is printed whenever the method is called:

Click here to view code image

```
May 17, 2016 10:57:59 AM Item hashCode
FINER: ENTRY
```

To achieve this, we do the following:

1. Load the bytecodes in the class file.
2. Locate all methods.
3. For each method, check whether it has a
   `LogEntry`
   annotation.
4. If it does, add the bytecodes for the following instructions at the beginning of the method:
   Click here to view code image

```
ldc
loggerName

invokestatic
  java/util/logging/Logger.getLogger:(Ljava/lang/String;)Ljava/util/logging/Logger;
ldc
className

ldc
methodName

invokevirtual
  java/util/logging/Logger.entering:(Ljava/lang/String;Ljava/lang/String;)V
```

Inserting these bytecodes sounds tricky, but ASM makes it fairly straightforward. We don't describe the process of analyzing and inserting bytecodes in detail. The important point is that the program in **Listing 8.9** edits a class file and inserts a logging call at the beginning of the methods annotated with the

`LogEntry`

annotation.

For example, here is how you add the logging instructions to

`Item.java`

in **Listing 8.10**, where

asm

is the directory into which you installed the ASM library:

Click here to view code image

```
javac set/Item.java
javac -classpath .:asm/\* bytecodeAnnotations/EntryLogger.java
java -classpath .:asm/\* bytecodeAnnotations.EntryLogger set.Item
```

Try running

```
javap -c set.Item
```

before and after modifying the

`Item`

class file. You can see the inserted instructions at the beginning of the

`hashCode`

,

`equals`

, and

`compareTo`

methods.

Click here to view code image

```
public int hashCode();
  Code:
  0: ldc #85; // String global
  2:   invokestatic    #80;
    // Method
    // java/util/logging/Logger.getLogger:(Ljava/lang/String;)Ljava/util/logging/Logger;
  5:   ldc     #86; //String Item
  7:   ldc     #88; //String hashCode
  9:   invokevirtual   #84;
    // Method java/util/logging/Logger.entering:(Ljava/lang/String;Ljava/lang/String;)V
  12:  bipush  13
  14:  aload_0
  15:  getfield        #2; // Field description:Ljava/lang/String;
  18:  invokevirtual   #15; // Method java/lang/String.hashCode:()I
  21:  imul
  22:  bipush  17
  24:  aload_0
  25:  getfield        #3; // Field partNumber:I
  28:  imul
  29:  iadd
  30:  ireturn
```

The

`SetTest`

program in **Listing 8.11** inserts

`Item`

objects into a hash set. When you run it with the modified class file, you will see the logging messages.

Click here to view code image

```
May 17, 2016 10:57:59 AM Item hashCode
FINER: ENTRY
May 17, 2016 10:57:59 AM Item hashCode
FINER: ENTRY
May 17, 2016 10:57:59 AM Item hashCode
FINER: ENTRY
May 17, 2016 10:57:59 AM Item equals
FINER: ENTRY
[[description=Toaster, partNumber=1729], [description=Microwave, partNumber=4104]]
```

Note the call to

`equals`

when we insert the same item twice.

This example shows the power of bytecode engineering. Annotations are used to add directives to a program, and a bytecode editing tool picks up the directives and modifies the virtual machine instructions.

Listing 8.9

bytecodeAnnotations/EntryLogger.java

Click here to view code image

```
 1 package bytecodeAnnotations;
 2
 3 import java.io.*;
 4 import java.nio.file.*;
 5
 6 import org.objectweb.asm.*;
 7 import org.objectweb.asm.commons.*;
 8
 9 /**
10  * Adds "entering" logs to all methods of a class that have the LogEntry annotation.
11  * @version 1.21 2018-05-01
12  * @author Cay Horstmann
13  */
14 public class EntryLogger extends ClassVisitor
15 {
16    private String className;
17
18    /**
19     * Constructs an EntryLogger that inserts logging into annotated methods of a given class.
20     */
21    public EntryLogger(ClassWriter writer, String className)
22    {
23       super(Opcodes.ASM5, writer);
24       this.className = className;
25    }
26
27    public MethodVisitor visitMethod(int access, String methodName, String desc,
```

```
28          String signature, String[] exceptions)
29    {
30       MethodVisitor mv = cv.visitMethod(access, methodName, desc, signature, exceptions);
31       return new AdviceAdapter(Opcodes.ASM5, mv, access, methodName, desc)
32       {
33          private String loggerName;
34
35          public AnnotationVisitor visitAnnotation(String desc, boolean visible)
36          {
37             return new AnnotationVisitor(Opcodes.ASM5)
38                {
39                   public void visit(String name, Object value)
40                   {
41                      if (desc.equals("LbytecodeAnnotations/LogEntry;")
42                            && name.equals("logger"))
43                         loggerName = value.toString();
44                   }
45                };
46          }
47
48          public void onMethodEnter()
49          {
50             if (loggerName != null)
51             {
52                visitLdcInsn(loggerName);
53                visitMethodInsn(INVOKESTATIC, "java/util/logging/Logger", "getLogger",
54                   "(Ljava/lang/String;)Ljava/util/logging/Logger;", false);
55                visitLdcInsn(className);
56                visitLdcInsn(methodName);
57                visitMethodInsn(INVOKEVIRTUAL, "java/util/logging/Logger", "entering",
58                   "(Ljava/lang/String;Ljava/lang/String;)V", false);
59                loggerName = null;
60             }
```

```
61            }
62         };
63   }
64
65   /**
66    * Adds entry logging code to the given class.
67    * @param args the name of the class file to patch
68    */
69   public static void main(String[] args) throws IOException
70   {
71      if (args.length == 0)
72      {
73         System.out.println("USAGE: java bytecodeAnnotations.EntryLogger classfile");
74         System.exit(1);
75      }
76      Path path = Path.of(args[0]);
77      var reader = new ClassReader(Files.newInputStream(path));
78      var writer = new ClassWriter(
79         ClassWriter.COMPUTE_MAXS | ClassWriter.COMPUTE_FRAMES);
80      var entryLogger = new EntryLogger(writer,
81         path.toString().replace(".class", "").replaceAll("[/\\\\]", "."));
82      reader.accept(entryLogger, ClassReader.EXPAND_FRAMES);
83      Files.write(Path.of(args[0]), writer.toByteArray());
84   }
85 }
```

Listing 8.10

set/Item.java

Click here to view code image

```
 1 package set;
 2
 3 import java.util.*;
 4 import bytecodeAnnotations.*;
 5
 6 /**
 7  * An item with a description and a part number.
 8  * @version 1.01 2012-01-26
 9  * @author Cay Horstmann
10  */
11 public class Item
12 {
13    private String description;
14    private int partNumber;
15
16    /**
17     * Constructs an item.
18     * @param aDescription the item's description
19     * @param aPartNumber the item's part number
20     */
21    public Item(String aDescription, int aPartNumber)
22    {
23       description = aDescription;
24       partNumber = aPartNumber;
25    }
26
27    /**
28     * Gets the description of this item.
29     * @return the description
30     */
31    public String getDescription()
```

```
32    {
33       return description;
34    }
35
36    public String toString()
37    {
38       return "[description=" + description + ", partNumber=" + partNumber + "]";
39    }
40
41    @LogEntry(logger = "com.horstmann")
42    public boolean equals(Object otherObject)
43    {
44       if (this == otherObject) return true;
45       if (otherObject == null) return false;
46       if (getClass() != otherObject.getClass()) return false;
47       var other = (Item) otherObject;
48       return Objects.equals(description, other.description) && partNumber == other.partNumber;
49    }
50
51    @LogEntry(logger = "com.horstmann")
52    public int hashCode()
53    {
54       return Objects.hash(description, partNumber);
55    }
56 }
```

Listing 8.11

set/SetTest.java

Click here to view code image

```
 1 package set;
 2
 3 import java.util.*;
 4 import java.util.logging.*;
 5
 6 /**
 7  * @version 1.03 2018-05-01
 8  * @author Cay Horstmann
 9  */
10 public class SetTest
11 {
12    public static void main(String[] args)
13    {
14       Logger.getLogger("com.horstmann").setLevel(Level.FINEST);
15       var handler = new ConsoleHandler();
16       handler.setLevel(Level.FINEST);
17       Logger.getLogger("com.horstmann").addHandler(handler);
18
19       var parts = new HashSet<Item>();
20       parts.add(new Item("Toaster", 1279));
21       parts.add(new Item("Microwave", 4104));
22       parts.add(new Item("Toaster", 1279));
23       System.out.println(parts);
24    }
25 }
```

8.7.2 Modifying Bytecodes at Load Time

In the last section, you saw a tool that edits class files. However, it can be cumbersome to add yet another tool into the build process. An attractive alternative is to defer the bytecode engineering until *load time*, when the class loader loads the class.

The *instrumentation API* has a hook for installing a bytecode transformer. The transformer must be installed before the `main` method of the program is called. You can meet this requirement by defining an *agent*, a library that is loaded to monitor a program in some way. The agent code can carry out initializations in a `premain` method.

Here are the steps required to build an agent:

1. Implement a class with a method

   Click here to view code image

   ```
   public static void premain(String arg, Instrumentation instr)
   ```

   This method is called when the agent is loaded. The agent can get a single command-line argument, which is passed in the `arg` parameter. The `instr` parameter can be used to install various hooks.
2. Make a manifest file `EntryLoggingAgent.mf`

that sets the

`Premain-Class`

attribute, for example:

Click here to view code image

```
Premain-Class: bytecodeAnnotations.EntryLoggingAgent
```

3. Package the agent code and the manifest into a JAR file:

   Click here to view code image

```
javac -classpath .:asm/\* bytecodeAnnotations/EntryLoggingAgent.java
jar cvfm EntryLoggingAgent.jar bytecodeAnnotations/EntryLoggingAgent.mf \
   bytecodeAnnotations/Entry*.class
```

To launch a Java program together with the agent, use the following command-line options:

Click here to view code image

```
java -javaagent:AgentJARFile=agentArgument . . .
```

For example, to run the

`SetTest`

program with the entry logging agent, call

Click here to view code image

```
javac set/SetTest.java
java -javaagent:EntryLoggingAgent.jar=set.Item -classpath .:asm/\* set.SetTest
```

The `Item` argument is the name of the class that the agent should modify.

Listing 8.12 shows the agent code. The agent installs a class file transformer. The transformer first checks whether the class name matches the agent argument. If so, it uses the `EntryLogger` class from the preceding section to modify the bytecodes. However, the modified bytecodes are not saved to a file. Instead, the transformer returns them for loading into the virtual machine (see Figure 8.3). In other words, this technique carries out "just in time" modification of the bytecodes.

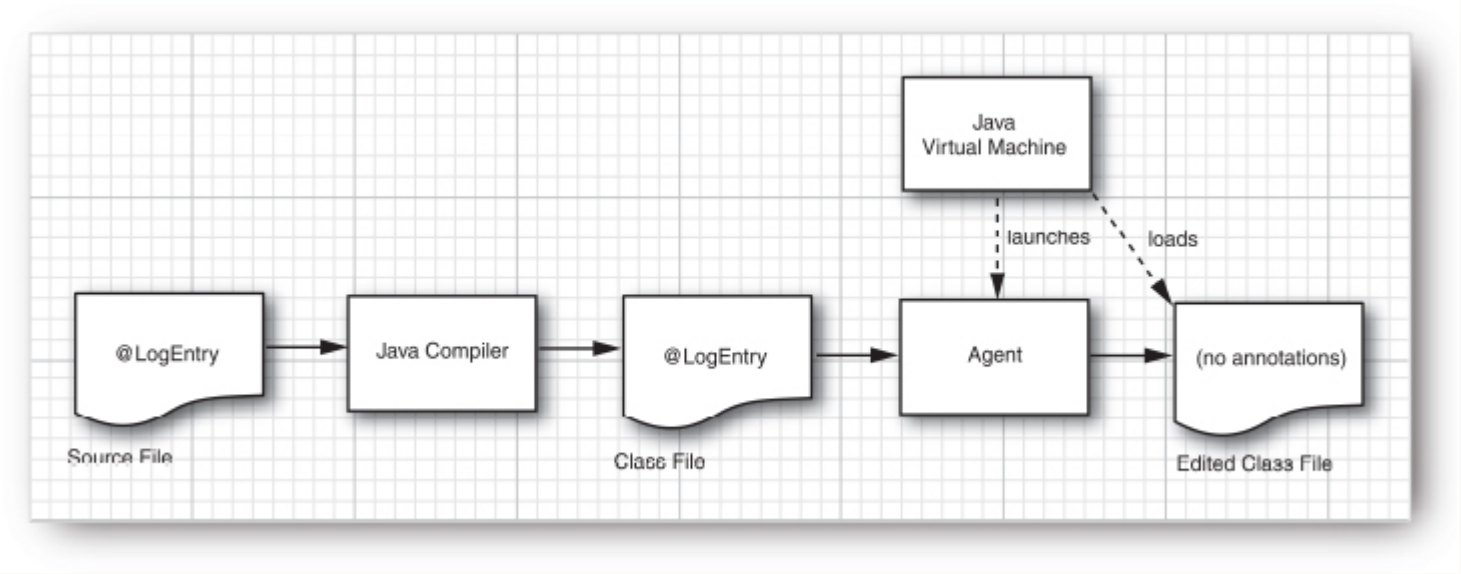


Figure 8.3 Modifying classes at load time

Listing 8.12

bytecodeAnnotations/EntryLoggingAgent.java

Click here to view code image

```
1 package bytecodeAnnotations;
2
3 import java.lang.instrument.*;
4
5 import org.objectweb.asm.*;
```

```
 6
 7 /**
 8  * @version 1.11 2018-05-01
 9  * @author Cay Horstmann
10  */
11 public class EntryLoggingAgent
12 {
13    public static void premain(final String arg, Instrumentation instr)
14    {
15       instr.addTransformer(new ClassFileTransformer()
16          {
17             public byte[] transform(ClassLoader loader, String className, Class<?> cl,
18                   ProtectionDomain pd, byte[] data) throws IllegalClassFormatException
19             {
20                if (!className.replace("/", ".").equals(arg)) return null;
21                var reader = new ClassReader(data);
22                var writer = new ClassWriter(
23                   ClassWriter.COMPUTE_MAXS | ClassWriter.COMPUTE_FRAMES);
24                var el = new EntryLogger(writer, className);
25                reader.accept(el, ClassReader.EXPAND_FRAMES);
26                return writer.toByteArray();
27             }
28          });
29    }
30 }
```

In this chapter, you have learned how to

- Add annotations to Java programs
- Design your own annotation interfaces

- Implement tools that make use of the annotations

You have seen three technologies for processing code: scripting, compiling Java programs, and processing annotations. The first two were quite straightforward. On the other hand, building annotation tools is undeniably complex and not something that most developers will need to tackle. This chapter gave you the background for understanding the inner workings of the annotation tools you will encounter, and perhaps piqued your interest in developing your own tools.

The following chapter discusses the Java Platform Module System, the key feature of Java 9 that is important for moving the Java platform forward.

Previous chapter
< Chapter 7. Internationalization

Next chapter
Chapter 9. The Java Platform Module System >